A Review Gate Stops Wrong Memory Writes but Not Missed Updates: Candidate Selection in User-Controlled Long-Term Memory for LLM Assistants

[Author name]
Affiliation and e-mail to be added

October 15, 2026

Abstract

Assistants that keep long-term memory about their user increasingly ask for approval before they change what they believe. We ask what such an approval gate does, and does not, protect. We study a local assistant (AURA) in which a language model proposes add, update, retire and forget operations, changes to existing memories wait for the user, and old values are kept as history. In simulation (60 persona runs per cell) the gate removes 94–98% of the true facts that automatic application destroys when the extractor errs, at 20–33 review prompts per 100 turns. It does not reduce stale facts: the extractor can only change memories that are shown in its prompt, and with default candidate selection the memory to be changed is shown for only 0.690 of change requests when 350 unrelated memories are stored, and for 0.286 when the change shares a message with other sentences. A candidate selection that searches sentence by sentence without a minimum score raises the latter to 0.866. On two real Gemini models (12 personas each) the change target was missing from the prompt in 25 and 22 of 136 change turns; with the new selection it was never missing, stale facts per persona fell from about 1 to 0 under the gate (sign test $p = 0.002$ and 0.002), and F1 rose by +0.077 and +0.057, at about 35% more prompt tokens and no significant increase in review prompts. A rule that lets a rejected change return after the user says something new gave no measurable benefit with real models. On 30 LongMemEval knowledge-update questions the gain with the gate (+0.167) is not significant ($p = 0.063$), and the memory system stays below full-context answering. The evaluation harness and all result files are provided.

Index Terms— long-term memory, conversational assistants, large language models, human-in-the-loop, memory update, evaluation.

1 Introduction

Large language model (LLM) assistants that remember facts about their user across conversations must decide, after every message, whether to add a fact, change one they already hold, or drop one. Mistakes are costly in two different ways. A *wrong write* replaces or removes something that was true ("moved to Pune" overwrites the true employer), and a *missed update* leaves an outdated value active after the user has said that it changed. A growing line of work treats wrong writes as a governance problem and proposes checks before a memory is stored [1]. A direct form of such a check is to ask the user: changes to existing memories become proposals that the user accepts or rejects, and old values are kept as history.

This paper starts from a simple observation about where such a gate sits. It acts on the operations that the extractor model *returns*. An update that the extractor never produces cannot be proposed, and therefore cannot be approved. In a retrieval-based design the extractor can change only the memories that are placed in its prompt, so the choice of these candidates bounds how many updates are possible at all. We therefore ask three questions about a user-controlled memory: (Q1) How much of the damage of extractor errors does the gate remove, and at what cost in user prompts? (Q2) How often is the memory that a change refers to visible to the extractor, and how does this depend on the size of the store and on how the candidates are chosen? (Q3) Does a better candidate selection translate into fewer stale facts with real models, and what does it cost?

We study these questions in AURA, a local assistant with a SQLite memory store, local sentence embeddings and a Gemini extractor, using the assistant's real server code throughout. Our contributions are:

1. We separate wrong writes from missed updates and measure both under a controlled error model. The gate cuts destroyed true facts by 94–98% but leaves stale facts essentially unchanged (Sec. 5).

2. We measure *reachability*, the share of change requests whose target memory is in the extractor's prompt, over store sizes from 0 to 350 and for single-sentence and mixed messages, and show that the default selection becomes the bottleneck (Sec. 6).

3. We describe two small switches, a candidate selection that searches per sentence without a minimum score

(F2) and a bounded re-proposal rule (F1), and evaluate them in an ablation in simulation and with two real Gemini models, including their cost. F2 carries the effect; F1 does not show a measurable benefit with real models (Sec. 7–8).

4. We report an external check on LongMemEval knowledge-update questions and state clearly what it does and does not show (Sec. 9), and we provide the harness, the personas generator and all raw results.

We deliberately keep claims narrow. The personas are synthetic and English, the real-model samples are small, the user is simulated, and no user study was run (Sec. 11).

2 Related Work

Memory for conversational agents and its benchmarks. Mem0 describes an architecture that extracts, consolidates and retrieves salient facts from conversations and evaluates it on LoCoMo [2], a benchmark of very long agent–agent conversations with question answering, summarisation and multimodal tasks [3]. LongMemEval benchmarks chat assistants on five long-term memory abilities, among them reasoning about updated facts, and splits memory design into indexing, retrieval and reading [4]. Our work uses LongMemEval's knowledge-update questions as an external check; the benchmark does not label which proposed memory changes are right, which is why our gate there uses an oracle approximation (Sec. 9).

Updating and governing memory. Lam et al. propose a governance framework (SSGM) that separates memory updates from task execution and applies consistency checks and permission controls before anything is stored [1]. Patel studies the *memory-update gap*, the loss of accuracy on questions about facts that changed, and addresses it by reinforcement-learning fine-tuning of a small model [5]. Xiong et al. show that adding and deleting stored experiences changes agent behaviour and that errors can compound through retrieval [6], and Xiao et al. find that errors entering through memory updates persist longer than errors entering through user questions [7]. Zhang et al. place a learned gate between memory store and model that decides which *retrieved* memories are admitted [8]. Our setting differs in three ways: the gate here is a user decision on *writes*, the problem we isolate is upstream of the gate (what the extractor can see), and the remedy is inference-time candidate selection rather than training. Chu surveys security threats to agents across layers and notes that current benchmarks rarely test failures that span sessions [9].

Scope. The works above were read at the level of their arXiv abstract pages for this paper, not in full; their detailed claims should be checked before they are relied upon. We make no claim about priority over any of them.

3 System: User-Controlled Memory in AURA

3.1 Memory store and operations

AURA keeps memories in SQLite. A memory has text, a category (preference, personal, people, work, plan, other), a status (`active`, `superseded` or `outdated`), a pointer to the memory that replaced it, a source and its conversation, and timestamps. After each user message a Gemini model (default `gemini-3.5-flash-lite`) receives the message and a list of candidate memories and returns operations: *add*, *update* (replace a value), *retire* (the fact no longer holds), or *forget*. An update keeps the old row as `superseded` with a link to its successor; retire marks the old row `outdated`. The extractor can refer only to memory identifiers that appear in its prompt.

3.2 The review gate

When "ask before changing memories" is on (the default), *add* operations are applied directly and every operation that touches an existing memory becomes a stored *proposal* (action, target memory, proposed text, conversation). The user accepts or rejects it; nothing changes until then. We call this policy **gate**. The comparison policies are **append** (only additions; a changed fact is stored as a second fact), **auto** (all operations applied at once), and **gate all** (also new facts wait for approval; emulated in the harness, not an AURA setting).

3.3 Candidate selection

The prompt contains candidate memories chosen from the store. By default these are the 12 best search matches for the user's message plus the 8 most recently created memories. Search combines the cosine similarity of MiniLM sentence embeddings [10] (`all-MiniLM-L6-v2`, run locally) and a lexical overlap score, $0.85 \cdot \cos + 0.15 \cdot \text{lex}$, and drops matches below a minimum score of 0.25. We evaluate four modes, selected by an environment variable:

- **default**: as above;
- **no minimum**: the same, without the 0.25 cut-off;
- **sentence**: the message is split into sentences and each is searched on its own, so that an unrelated sentence does not dilute the query;
- **adaptive** (F2): sentence search without minimum score, and a larger number of matches (up to 24) when the message contains a change cue (words such as "moved", "stopped", "no longer", "forget").

Using F2 enlarges the prompt (Sec. 8); the minimum score exists to keep unrelated memories out (Fig. 3).

Table 1: What is real and what is simulated.

Part	Sim. runs	Real runs
Store, operations, proposals, search	AURA code	AURA code
Sentence embeddings	MiniLM	MiniLM
Extractor	simulated, rate e	Gemini
Reviewing user	simulated, slip u	simulated, $u = 0$
Conversations	scripted personas	same personas

3.4 Re-proposal (F1)

A rejected proposal normally ends the matter. With F1 a rejected change may be proposed again, at most N times, but only after the user has said something new about it (we use $N = 2$). The aim is to recover from a rejection that was a slip without nagging the user.

4 Evaluation Method

All experiments run AURA's real server code, real SQLite schema and real local embedding model on a throw-away database. What is real and what is simulated is listed in Table 1.

4.1 Personas and ground truth

A persona has 8 of 12 fact types (city, employer, dog, sister, diet, gym time, language, car, project, coffee, hobby, birthday) and five sessions of about 23 user turns. Facts are stated, later updated ("I've just moved to Hyderabad"), retired ("I stopped going to the gym"), forgotten on request, repeated, and mixed with distractors that must not become memories (hypotheticals, questions, assistant suggestions, facts about friends). A store starts with 30 unrelated true background facts and gains 10 between sessions, imitating earlier use. The true state is known after every turn. In the ablation runs a restating turn ("Like I said, ...") follows each update, which is what F1 needs to act on.

4.2 Simulated extractor and user

The simulated extractor returns what a correct model would return for the turn, using only the memories shown in the prompt; a change whose target is not shown becomes an add, as a real model would do. Errors are injected at rate e: a correct operation is dropped, a distractor becomes a false memory, or a true memory shown to the model is retired or overwritten with a false value. All policies face the same random draws. The simulated user reviews proposals at the end of a session, accepts correct ones and rejects wrong ones, and errs with probability u (default 0.05).

4.3 Metrics

F1 compares the persona's active memories with the true facts. **Stale** counts active memories holding a value the user gave earlier that is no longer true. **False** counts active memories the user never stated about themselves. **Destroyed** counts true memories, persona and background, that ended up superseded, outdated or deleted by a wrong change; F1 does not see the background memories, so this is the safety metric. **Prompts/100** is the number of review cards per 100 user turns. **Reachability** is the share of change requests whose target memory is among the extractor's candidates. **Prompt tokens** are summed over a persona run.

4.4 Real-model runs and scoring

Real runs use Gemini as the extractor for 12 personas, one seed, with the simulated user as an oracle ($u = 0$), for the policies auto and gate and four variants (baseline, F1, F2, F1+F2). They were repeated for baseline and F1+F2 with a second model (`gemini-3.1-flash-lite`). Because models word memories freely, a memory is scored by value: the automatic scorer maps each active memory to a persona fact and a label (true, stale, ended, hypothetical, third party, other). The author hand-labelled every memory of the baseline and F1+F2 runs (195 and 175 memories). There were 3324 extraction calls and 0 failed.

4.5 Statistics

Differences between variants are paired by persona and seed. We report the mean difference with a 95% bootstrap interval (4000 resamples, fixed seed) and an exact two-sided sign test on the non-tied pairs. No correction for multiple tests is applied; the tests run are 210 (simulated ablation) and 48 (real runs), so isolated p-values near 0.03 should not be read as findings. Reachability is reported with Wilson intervals [11], and agreement between hand and automatic labels with Cohen's κ [12]; since there is a single annotator this measures the scorer against the annotator, not human–human agreement.

5 Results: What the Gate Protects

Fig. 1 and Table 2 compare the four policies over extractor error rates e (60 persona runs per cell: 20 personas, 3 seeds).

Wrong writes. Under auto-apply the extractor's errors destroy true facts: 0.78, 1.60, 2.83 and 4.60 per persona at $e = 0.05, 0.1, 0.2, 0.3$. The gate reduces this to 0.05, 0.03, 0.18 and 0.25, i.e. by 94–98%. The remainder comes from the simulated user's slips. The price is 20–33 review prompts per 100 turns; requiring approval for new facts as well removes false new facts but needs about 2.7 times as many prompts as the gate.

Missed updates. The gate does not help here. At $e = 0$ the append-only policy leaves 5.00 stale facts per persona by construction, and the gate still leaves 1.80, slightly more than auto-apply (1.57); across the error sweep the gate leaves between 1.8 and 2.2 stale facts,

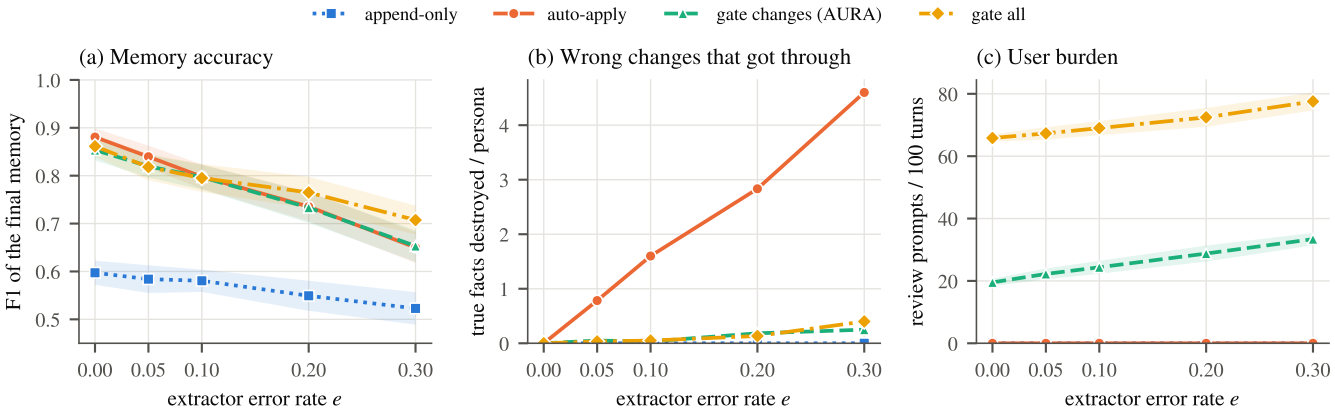


Figure 1: Policies under extractor errors (simulated, shaded: 95% CI). (a) F1. (b) True facts destroyed by wrong changes. (c) Review prompts per 100 turns.

Table 2: Review policies versus extractor error rate e (simulated, user slip $u = 0.05$, mean per persona).

e	policy	F1	stale	false	destroyed	prompts/100
0	append-only	0.597 ± 0.025	5.00	0.00	0.00	0.0
	auto-apply	0.881 ± 0.018	1.57	0.00	0.00	0.0
	gate changes	0.854 ± 0.022	1.80	0.00	0.00	19.5
	gate all	0.861 ± 0.023	1.62	0.00	0.00	65.8
0.1	append-only	0.581 ± 0.023	4.58	0.33	0.00	0.0
	auto-apply	0.798 ± 0.026	1.68	0.43	1.60	0.0
	gate changes	0.797 ± 0.026	1.85	0.33	0.03	24.4
	gate all	0.795 ± 0.029	1.83	0.02	0.05	69.0
0.3	append-only	0.523 ± 0.034	3.87	1.20	0.00	0.0
	auto-apply	0.650 ± 0.032	1.92	1.33	4.60	0.0
	gate changes	0.653 ± 0.033	2.18	1.22	0.25	33.4
	gate all	0.707 ± 0.030	1.98	0.07	0.40	77.6

Table 3: Gate under user slips (simulated, $e = 0.1$).

slip u	F1	stale	wrong changes applied	prompts/100
0	0.808 ± 0.026	1.75	0.02	24.5
0.1	0.783 ± 0.028	2.00	0.28	24.4
0.2	0.757 ± 0.027	2.33	0.42	24.0
0.3	0.732 ± 0.029	2.52	0.62	24.1

auto-apply between 1.6 and 1.9. F1 of the gate is below auto-apply at $e = 0$ (0.854 vs. 0.881) and about equal at higher e. The gate thus buys safety at a small cost in F1, and does nothing for stale facts.

User slips. Table 3 varies the simulated user's slip rate at $e = 0.1$. F1 falls from 0.808 to 0.732 and stale facts rise from 1.75 to 2.52 as u goes from 0 to 0.3: a gate is only as good as the review.

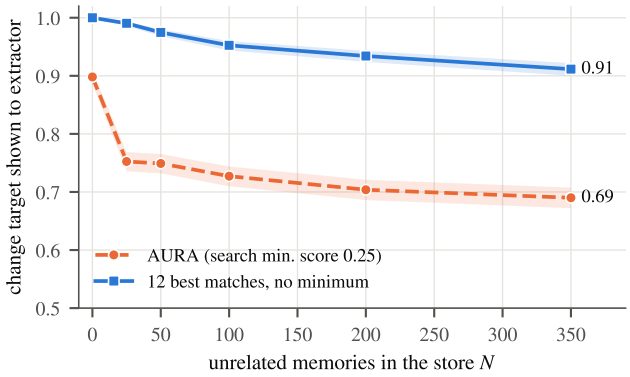


Figure 2: Share of change requests whose target memory is shown to the extractor, versus store size N (shaded: Wilson 95% CI).

6 Results: What the Extractor Can See

For each of 30 stores per size we generated update, retire and forget requests (2610 in all) and checked whether the memory the request refers to is among the extractor's candidates. The store holds N unrelated memories besides the ones the persona has stated.

Reachability falls from 0.898 with an empty store to 0.690 at $N = 350$ (Fig. 2, Table 4). Even with an empty store about one change in ten is out of reach because the 0.25 minimum drops weak matches. Without the

Table 4: Reachability for single-sentence changes (95% Wilson interval).

N	shown (default)	shown (no minimum score)
0	0.898 [0.886, 0.909]	1.000 [0.999, 1.000]
25	0.752 [0.736, 0.769]	0.990 [0.986, 0.994]
50	0.749 [0.732, 0.765]	0.975 [0.968, 0.980]
100	0.727 [0.710, 0.744]	0.952 [0.944, 0.960]
200	0.704 [0.686, 0.721]	0.934 [0.924, 0.943]
350	0.690 [0.672, 0.707]	0.911 [0.900, 0.922]

Table 5: Reachability by selection mode (mean number of candidates in parentheses).

N	turn type	default	no min.	sentence	adaptive
0	single sentence	0.898 (8.2)	1.000 (11.4)	0.898 (8.2)	1.000 (11.4)
	mixed turn	0.747 (8.1)	0.998 (11.4)	0.898 (8.2)	1.000 (11.9)
350	single sentence	0.690 (11.3)	0.911 (19.8)	0.690 (11.3)	0.936 (28.2)
	mixed turn	0.286 (8.3)	0.607 (19.8)	0.690 (11.3)	0.866 (28.1)

minimum the figure is 0.911 at $N = 350$. The recent-memory slots help only at small N.

Table 5 compares the modes, with the mean number of candidates in parentheses. When the change shares the message with two unrelated sentences, which is common in natural conversation, default selection reaches only 0.286 of changes at $N = 350$, because the embedding of the whole message is dominated by the unrelated content. Removing the minimum score gives 0.607; per-sentence search alone 0.690; and the adaptive mode 0.866, at 28.1 candidates on average instead of 8.3. Per-sentence search makes a mixed message behave like a single sentence. The minimum score is not free to remove, because it also suppresses unrelated matches (Fig. 3), and a larger prompt costs tokens.

7 Results: Ablation of the Two Switches

7.1 Simulated

Table 6 and Fig. 4 give the paired ablation under the gate (error sweep, $u = 0.05$, restating turns on). F2 reduces stale facts per persona at $e = 0.1$ from 1.83 to 0.65 and F1+F2 to 0.58; the F2 effects are significant ($p < 0.001$) in every cell shown. F1 alone moves F1 by +0.012 at $e = 0$ ($p = 0.031$), and is not significant at $e = 0.1$ and 0.2. With 210 tests that single p-value would not survive a correction. F2 raises the review load by about 5 prompts per 100 turns at $e = 0.1$ because more changes are discovered and proposed.

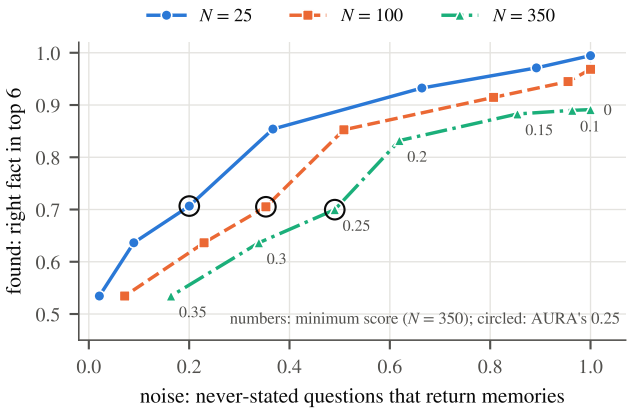


Figure 3: Retrieval trade-off of the minimum score: share of stored facts found in the top six versus share of never-stated questions that still return memories. Circled: AURA's 0.25.

8 Results: Real Gemini Extractors

Table 7 and Fig. 5 give the same ablation with a real extractor (12 personas, one seed, oracle user). Table 8 repeats baseline and F1+F2 with a second model.

Reachability on real conversations. With the baseline selection, the target memory was missing from the prompt on 25 of 136 change turns (first model; both policies together) and 22 of 136 (second model). With F1+F2 it was missing on 0 and 0. This is the mechanism of Sec. 6 observed with real conversations.

Stale facts and F1. Under the gate, stale facts per persona fell from 1.00 to 0.00 (first model; $p = 0.002$) and from 1.08 to 0.00 (second; $p = 0.002$), and F1 rose from 0.905 to 0.982 (+0.077, $p = 0.021$) and from 0.845 to 0.901 (+0.057, $p = 0.002$). F2 alone reached 0.33 stale facts ($p = 0.021$) and F1 of 0.953 ($p = 0.146$, not significant); the benefit of adding F1 to F2 was not tested separately and is within what 12 personas can resolve. F1 alone changed nothing measurable (stale 1.08, $p = 1.000$): on real models F1 is not supported. Under auto-apply the picture is the same (stale 1.00 to 0.00, $p = 0.002$).

Safety on real models. No injected errors exist in real runs, so we count what the models did. Across all auto-apply persona runs (72), 7 true background facts were destroyed by the model's own operations (5 of them in the two baseline runs); across the 72 gate runs the count was 0. With the oracle user, no correct change was rejected, no wrong change was accepted and 0 wrong changes were applied. Because the user is perfect and the counts are small, this is a consistency check on the simulation, not a measurement of real users.

Cost. Prompt tokens per persona rose from about 13.4k to 18.1k (+35%, $p < 0.001$), and the number of prompts per 100 turns from 23.3 to 24.9 (first model, $p = 0.289$) and from 24.5 to 26.0 (second, $p = 0.180$), not significant.

Table 6: Ablation under the gate (simulated). In parentheses: difference to the baseline and sign-test p (60 pairs).

e	variant	F1 (diff., p)	stale (diff., p)	prompts/100
0	baseline	0.854	1.80	17.6
	+F1	0.865 (+0.012, 0.031)	1.70 (−0.10, 0.031)	18.3
	+F2	0.950 (+0.096, <0.001)	0.40 (−1.40, <0.001)	24.3
	F1+F2	0.967 (+0.114, <0.001)	0.27 (−1.53, <0.001)	25.0
0.1	baseline	0.817	1.83	23.7
	+F1	0.821 (+0.005, 0.500)	1.80 (−0.03, 0.500)	23.9
	+F2	0.890 (+0.074, <0.001)	0.65 (−1.18, <0.001)	29.1
	F1+F2	0.901 (+0.084, <0.001)	0.58 (−1.25, <0.001)	29.4
0.2	baseline	0.737	1.98	28.6
	+F1	0.747 (+0.010, 0.250)	1.92 (−0.07, 0.250)	28.8
	+F2	0.776 (+0.039, <0.001)	1.22 (−0.77, <0.001)	32.2
	F1+F2	0.784 (+0.047, <0.001)	1.17 (−0.82, <0.001)	32.7

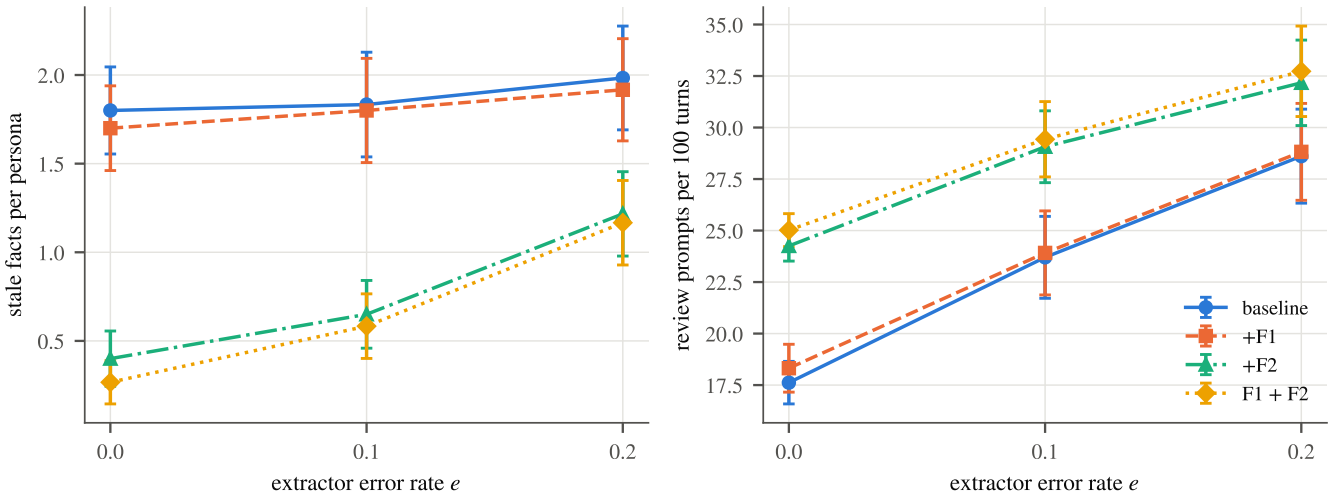


Figure 4: Gate policy, error sweep: stale facts and review prompts for baseline, F1, F2, F1+F2 (simulated).

Of the change prompts, 68 of 69 (first model) and 68 of 72 (second) in the F1+F2 runs were correct changes that the user accepted, against 56 of 65 and 55 of 68 for the baselines; this is consistent with the added prompts being updates that the baseline missed.

Other effects. Under auto-apply the F2-only variant stored 3 false facts across 12 personas where the baseline stored none, and F1+F2 stored none again; with these counts we cannot say whether this is an effect of F2 or noise.

8.1 Scorer validity

Table 9 shows the agreement of the automatic labels with the author's hand labels. Three of $195 + 175$ memories differ; in all three the automatic scorer labelled a correct statement of unemployment or finished project as "ended" or "other" while the annotator judged it true, so the automatic F1 is, if anything, slightly conservative on these three. These labels were kept as annotated. Since the scorer and the labels share one author and the same persona rules, the agreement shows consistency, not independence.

9 External Check: LongMemEval

LongMemEval's knowledge-update questions ask for the latest value of a fact that changed during the history [4]. We took 30 such questions of the "S" setting and played each with all evidence sessions plus three random distractor sessions (the same for every system) through AURA's extractor, store and search; answers come from the ten memories that AURA's search returns, and are graded by a Gemini judge written after the benchmark's rule (the official prompt is not reproduced) and by string match. All runs use `gemini-3.1-flash-lite`. There is no label for which memory changes are right, so the gate's reviewer is an oracle approximation: it accepts proposals made while reading an evidence session and rejects the rest. This gate figure is therefore an upper-bracket, not a measurement of real users. Baselines are answering with no memory and with the raw sessions as context.

Table 10 and Fig. 6 show that the gate system improves from 0.267 to 0.433 (0 questions worse, 5 better, $p = 0.063$), which is in the expected direction but not significant at this sample size; the systems without memory or with full context, which cannot be affected, differ by

Table 7: Real extractor `gemini-3.5-flash-lite`: 12 paired personas per cell. In parentheses: difference to the baseline and exact sign-test p.

policy	variant	F1	stale	false	prompts/100	prompt tokens ($\times 10^3$)
auto	baseline	0.908	1.00	0.00	0.0	13.2
	+F1	0.902 (−0.007, 1.000)	1.00 (+0.00, 1.000)	0.08 (+0.08, 1.000)	0.0	13.2
	+F2	0.943 (+0.035, 0.453)	0.00 (−1.00, 0.002)	0.25 (+0.25, 0.250)	0.0	18.0
	F1+F2	0.956 (+0.048, 0.109)	0.00 (−1.00, 0.002)	0.00 (+0.00, 1.000)	0.0	18.0
gate	baseline	0.905	1.00	0.00	23.3	13.4
	+F1	0.897 (−0.008, 0.625)	1.08 (+0.08, 1.000)	0.00 (+0.00, 1.000)	21.9	13.3
	+F2	0.953 (+0.048, 0.146)	0.33 (−0.67, 0.021)	0.08 (+0.08, 1.000)	24.2	18.2
	F1+F2	0.982 (+0.077, 0.021)	0.00 (−1.00, 0.002)	0.00 (+0.00, 1.000)	24.9	18.1

Table 8: Second extractor `gemini-3.1-flash-lite`: baseline versus F1+F2, 12 paired personas.

policy	variant	F1	stale	false	prompts/100	prompt tokens ($\times 10^3$)
auto	baseline	0.840	0.92	0.00	0.0	13.2
	F1+F2	0.892 (+0.053, 0.012)	0.00 (−0.92, <0.001)	0.00 (+0.00, 1.000)	0.0	18.1
gate	baseline	0.845	1.08	0.00	24.5	13.3
	F1+F2	0.901 (+0.057, 0.002)	0.00 (−1.08, 0.002)	0.00 (+0.00, 1.000)	26.0	18.4

Table 9: Agreement between automatic and hand labels (single annotator).

run	labelled memories	agreement	rate	Cohen's κ
baseline	195	193	0.990	0.975
F1+F2	175	174	0.994	0.981

Table 10: LongMemEval knowledge-update questions (judge-graded accuracy, $n = 30$). b/c: questions correct only in the baseline / only in the F1+F2 run.

system	baseline	F1+F2	difference	b/c	p
no memory	0.000	0.000	+0.000	0/0	1.000
full context	0.767	0.733	−0.033	1/0	1.000
append-only	0.400	0.333	−0.067	3/1	0.625
auto-apply	0.467	0.500	+0.033	1/2	1.000
gate (oracle reviewer)	0.267	0.433	+0.167	0/5	0.063

−0.033 between the two runs, which indicates run-to-run noise of that size. Auto-apply and append are not distinguishable between the runs. Full-context answering (0.733 in the second run) remains clearly above the gate system (0.433; 9 vs. 0 discordant questions, $p = 0.004$): a memory system trades accuracy for the properties it provides, such as small prompts, forgetting and user control. A run on all 78 knowledge-update questions would be needed for a firm statement.

10 Discussion

Where the guarantee ends. The gate provides a guarantee about what is *written*: nothing changes without approval. It provides none about what is *proposed*. In our experiments the second quantity depends on a component that looks like plumbing, the choice of candidate memories, and with default settings it is the main reason that stale facts remain. Evaluating a user-controlled memory by its write accuracy or by the number of approvals alone would miss this.

Design implications. (i) Report reachability alongside accuracy for any extractor-based memory. (ii) Search per sentence rather than per message when messages are mixed. (iii) Treat the minimum score as a trade-off: the unfiltered selection raises reachability (0.911 at $N = 350$) but admits unrelated memories, so the price of F2 is a larger prompt. (iv) Budget for more prompt tokens (about 35%) and slightly more review cards.

On F1. Letting a rejected change return only after new user input is cheap and conceptually neat, but we found no benefit with real models (the oracle user rejected no correct change in these runs), and only small, uncorrected-significance effects in simulation. Its value would have to be shown with real users who slip.

11 Threats to Validity

Synthetic and English personas. Personas, distractors and the wording of changes were written by the system's author. The change cues of F2 were designed after the reachability failures were seen on the same generator, so F2 may be tuned to it; the LongMemEval check is the only independent text and is small. **Small real samples.** 12 personas, one seed per cell and two models of the same family; exact sign tests with 12 pairs cannot resolve small effects, and several F1 results are inconclusive rather than null. **Simulated user.** The real runs use a perfect reviewer, the simulations a random slip. Real users may ignore, batch or rubber-stamp prompts; whether additional prompts are tolerated is open. No user study was done. **Single annotator and scorer.** Labels come from the author. **One embedding model.** Reachability depends on MiniLM and the 0.25 threshold. **LongMemEval.** Thirty questions, a judge that is not the official prompt, an oracle reviewer, and a single seed. **Multiple tests.** Reported

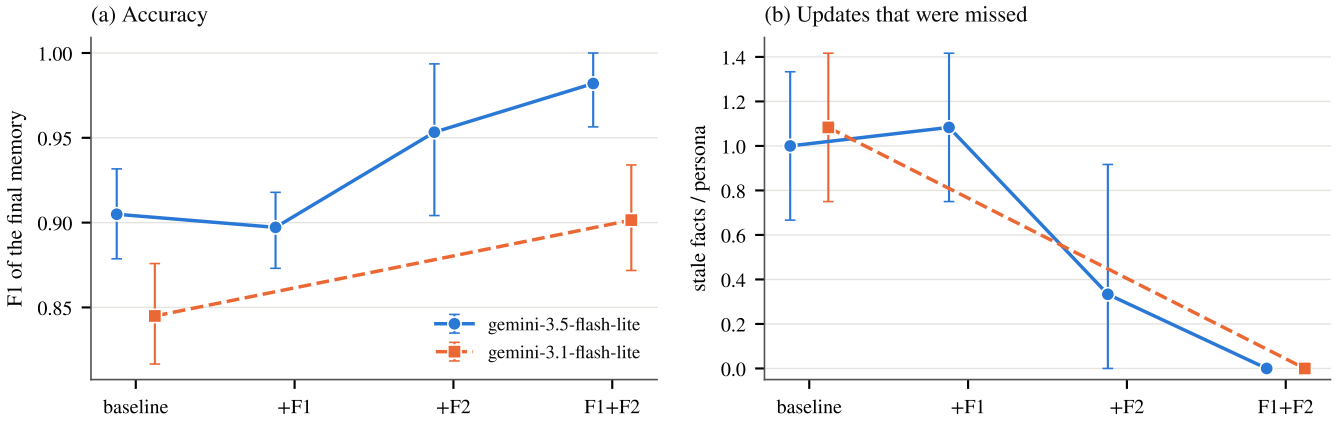


Figure 5: Gate policy with real Gemini extractors: (a) F1, (b) stale facts per persona; mean and 95% bootstrap interval over 12 personas.

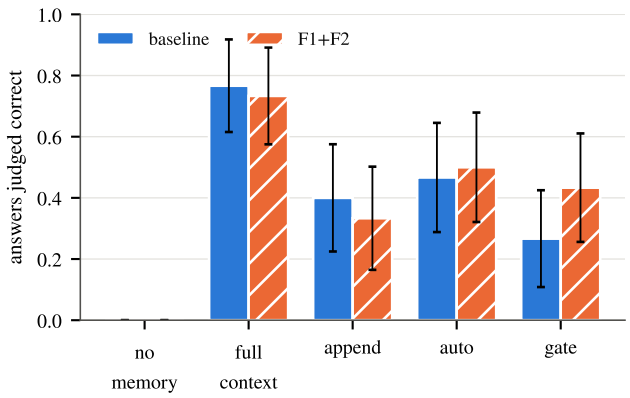


Figure 6: LongMemEval accuracy by system for baseline and F1+F2 selection (95% normal-approximation bars).

as counts; only the large effects are robust to correction.

Related work. Sources were read at abstract level.

12 Conclusion

A review gate on memory changes is effective against the errors it can see: in our experiments it removes almost all of the true facts that automatic application destroys. It does not address updates the extractor never proposes, and in a retrieval-based design these are governed by which memories the extractor is shown. Showing candidates by sentence, without a minimum score and with a larger set on a change cue, removed all observed misses for two real Gemini models and removed the stale facts, at roughly a third more prompt tokens and no significant increase in review prompts, while an external check on LongMemEval pointed in the same direction without reaching significance. The practical lesson for user-controlled memories is to measure and report what the extractor can reach. Evaluating with real users, in other languages and with larger real samples is the necessary next step; the evaluation code and every raw result file are provided so that others can repeat and extend it.

Data and Code Availability

The evaluation harness (`eval/`), the persona generator, all raw per-run files and the scripts that produce every table and figure of this paper (`eval/make_paper_assets.py`) are in the AURA project repository. Numbers in the text are generated from the result folders.

References

[1] Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework. *arXiv preprint arXiv:2603.11768*, 2026.

[2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[3] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.

[4] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[5] Vedant Patel. Supersede: Diagnosing and training the memory-update gap in LLM agents. *arXiv preprint arXiv:2606.27472*, 2026.

[6] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. *arXiv preprint arXiv:2505.16067*, 2025.

[7] Shuyao Xiao, Shengling Wang, Xuan Chen, Ke Chao, Ming Cui, Feifei Qian, Fanlin Meng, Chaoyang Mei, Chaoyong Jiang, Qi Ouyang, and Junxi Yi. When errors become memories: Causal pathway tracing in multi-turn memory-augmented LLMs. *arXiv preprint arXiv:2608.30198*, 2026.

[8] Jiawen Zhang, Kejia Chen, Jiachen Ma, Yangfan Hu, Lipeng He, Yechao Zhang, Jian Liu, Xiaohu Yang, Tianwei Zhang, and Ruoxi Jia. Beyond similarity: Trustworthy memory search for personal AI agents. *arXiv preprint arXiv:2606.06054*, 2026.

[9] Kexin Chu. A systematic survey of security threats and defenses in LLM-based AI agents: A layered attack surface framework. *arXiv preprint arXiv:2604.23338*, 2026.

[10] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *arXiv preprint arXiv:2002.10957*, 2020.

[11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[12] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.